

Your idea is **very suitable** for a DSA project because it focuses on **tracking data, searching, history management, and relationships** rather than requiring a large-scale web application. It also naturally allows you to demonstrate multiple data structures in meaningful ways.

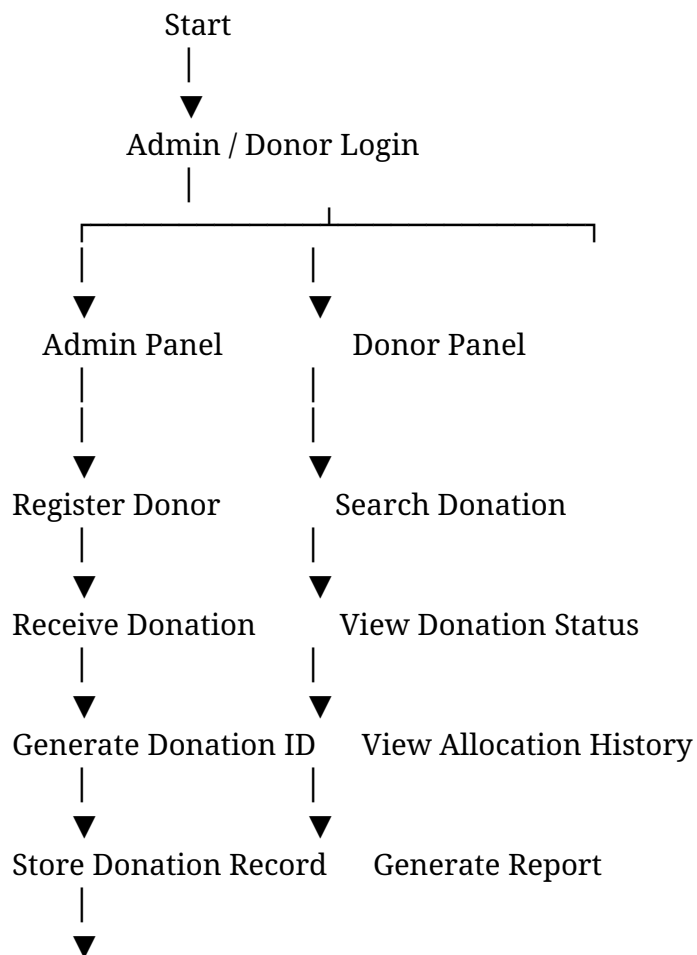
I would keep the project scope focused:

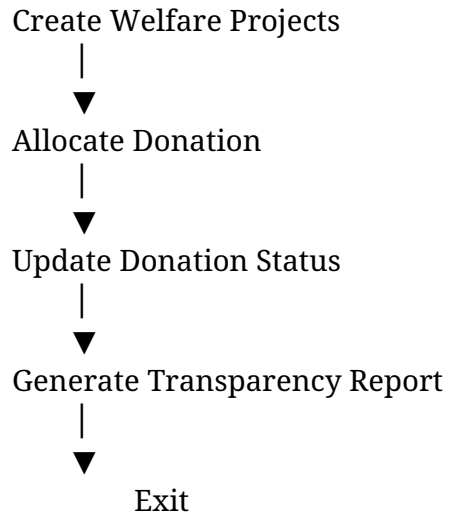
Donation Transparency & Accountability System (CLI Based)

A charity organization records donations, allocates funds to different welfare projects, and allows donors to track exactly where their donations have been utilized.

This is achievable within **2-5 days** while showcasing several DSA concepts.

Project Flow





Main Modules

1. Donor Management

Purpose

Store donor information.

Operations

- Register donor
- Edit donor
- Delete donor
- Search donor

Suggested Data Structure

Hash Table

Reason

Searching donors by CNIC or Donor ID should be $O(1)$.

Instead of

`vector<Donor>`

implement

Hash Table

Index

0

1

2

3

4

...

bucket -> linked list

Collision Resolution

Separate Chaining using Singly Linked List

This demonstrates

- Hashing
 - Linked List
-

2. Donation Management

Each donation contains

Donation ID

Donor ID

Amount

Date

Purpose

Current Status

Statuses

Received

Allocated

In Progress

Completed

Suggested Structure

Doubly Linked List

Reason

Admin frequently moves forward/backward through donations.

Supports

- insertion
- deletion
- traversal

3. Project Management

Example

Food Drive

Medical Camp

Mosque Renovation

Madrasa Scholarship

Water Well

Ramadan Ration

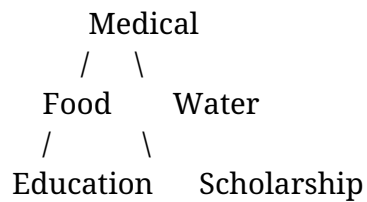
Suggested Structure

AVL Tree

Reason

Projects are searched frequently.

Balanced tree



Searching remains

$O(\log n)$

4. Donation Allocation

When admin allocates money

Donation #102

↓

Medical Camp

↓

Allocated

Every allocation becomes a transaction.

Suggested Structure

Queue

Reason

Allocations happen in order.

FIFO

Donation A

Donation B

Donation C

First received

↓

First processed

5. Expense Tracking

Suppose

Medical Camp receives

100,000

Expenses

Medicine

Doctor Fees

Ambulance

Food

Every expense becomes

Expense ID

Amount

Description

Date

Suggested Structure

Linked List

Because expenses are appended continuously.

6. Donation History

Every status update

Received

↓

Allocated

↓

Medicine Purchased

↓

Completed

should remain visible.

Suggested Structure

Stack

Reason

Latest activity appears first.

Example

Completed

Medicine Purchased

Allocated

Received

Undo feature can also be demonstrated.

7. Search Donation

User enters

Donation ID

System shows

Donor

Amount

Project

Current Status

Remaining Amount

History

Suggested Structure

Hash Table

Lookup is nearly $O(1)$.

8. Transparency Report

Example

Donation

↓

Allocation

↓

Expenses

↓

Balance

Example Output

Donation

Rs.10000

↓

Allocated

Medical Camp

↓

Medicine

Rs.3000

↓

Doctor

Rs.2000

↓

Remaining

Rs.5000

9. Statistics

Examples

Largest Donation

Smallest Donation

Total Donations

Average Donation

Project with Most Funding

Pending Donations

Suggested Structure

Max Heap

To find

Largest Donation

```
      200000
     /   \
    /     \
150000   120000
```

Min Heap

For smallest donation.

10. Relationship Between Projects

Suppose

Flood Relief

↓

Food

↓

Medicine

↓

Shelter

Represent dependency.

Suggested Structure

Graph

Adjacency List

Flood Relief

|

|— Food

|— Medicine

└— Shelter

Traversal

DFS

Display complete workflow.

BFS

Display level-wise activities.

This is optional but allows inclusion of graph algorithms in a meaningful way.

Overall Data Structure Mapping

Module	Best Data Structure	Why
Donor Database	Hash Table + Linked	Fast donor lookup

Module	Best Data Structure	Why
	List	
Donation Records	Doubly Linked List	Easy traversal and updates
Welfare Projects	AVL Tree	Balanced searching
Allocation Queue	Queue	FIFO processing
Expense Records	Singly Linked List	Sequential insertion
Status History	Stack	Latest updates first
Largest Donation	Max Heap	Efficient maximum retrieval
Smallest Donation	Min Heap	Efficient minimum retrieval
Project Dependencies	Graph	Show project relationships

Suggested Classes

Donor

Donation

Project

Expense

Allocation

HashTable

AVLTree

Queue

Stack

Heap

Graph

System

Example User Flow

Start

↓

Register Donor

↓

Donate Rs.5000

↓

Donation ID Generated

↓

Stored in Hash Table

↓

Donation added to Doubly Linked List

↓

Admin creates Project

↓

Project inserted in AVL Tree

↓

Donation placed in Allocation Queue

↓

Admin allocates donation

↓

Status pushed to Stack

↓

Expense added

↓

Transparency Report Generated

↓

Donor searches Donation ID

↓

System displays

Donation Details

Allocation Details

Expenses

Remaining Balance

Status History

↓

Exit

Example CLI Menu

```
=====
Donation Transparency System
=====
```

1. Register Donor
 2. Make Donation
 3. View Donor
 4. Create Welfare Project
 5. Allocate Donation
 6. Add Expense
 7. Search Donation
 8. View Transparency Report
 9. View Donation History
 10. Statistics
 11. Exit
-

Why this project is strong for a DSA course

This project avoids forcing every data structure into the design. Instead, each one has a clear purpose:

- **Hash Table** for constant-time lookup of donors and donations.
- **Doubly Linked List** for maintaining donation records with efficient insertion and traversal.
- **AVL Tree** for balanced storage and searching of welfare projects.
- **Queue** to process donation allocations in arrival order.
- **Stack** to maintain an audit trail of status changes.
- **Min/Max Heap** to retrieve smallest and largest donations efficiently.
- **Graph (optional extension)** to model project phases or dependencies and demonstrate BFS/DFS.

This keeps the project academically sound while remaining feasible for a third-semester student to complete within the allotted time. If you want to showcase more advanced algorithms such as **Dijkstra's, Prim's, or Kruskal's**, a practical

extension would be to model the organization's field offices and distribution routes as a weighted graph, using Dijkstra to find the shortest delivery route and Prim/Kruskal to design a minimum-cost network connecting offices. That extension is optional and can be added after the core system is complete.